JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Class ReentrantReadWriteLock.ReadLock

java.lang.Object
  java.util.concurrent.locks.ReentrantReadWriteLock.ReadLock

**All Implemented Interfaces:**

Serializable, Lock

**Enclosing class:**

ReentrantReadWriteLock

---

```
public static class ReentrantReadWriteLock.ReadLock
extends Object
implements Lock, Serializable
```

The lock returned by method `ReentrantReadWriteLock.readLock()`.

**Since:**

1.5

**See Also:**

Serialized Form

## *Constructor Summary*

**Constructors**

| Modifier | Constructor | Description |
|---|---|---|
| protected | **ReadLock**(ReentrantReadWriteLock) | Constructor for use by subclasses. |

## *Method Summary*

**All Methods** | **Instance Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| void | **lock**() | Acquires the read lock. |
| void | **lockInterruptibly**() | Acquires the read lock unless the current thread is interrupted. |
| Condition | **newCondition**() | Throws `UnsupportedOperationException` because `ReadLocks` do not support conditions. |

| | | |
|---|---|---|
| String | **toString**() | Returns a string identifying this lock, as well as its lock state. |
| boolean | **tryLock**() | Acquires the read lock only if the write lock is not held by another thread at the time of invocation. |
| boolean | **tryLock**(long timeout, TimeUnit unit) | Acquires the read lock if the write lock is not held by another thread within the given waiting time and the current thread has not been interrupted. |
| void | **unlock**() | Attempts to release this lock. |

**Methods declared in class Object**

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

## *Constructor Details*

### ReadLock

```
protected ReadLock(ReentrantReadWriteLock lock)
```

Constructor for use by subclasses.

**Parameters:**

`lock` - the outer lock object

**Throws:**

NullPointerException - if the lock is null

## *Method Details*

### lock

```
public void lock()
```

Acquires the read lock.

Acquires the read lock if the write lock is not held by any thread and returns immediately.

If the write lock is held by any thread or the fairness policy prohibits acquisition of the read lock at this time, then the current thread becomes disabled for thread scheduling purposes and lies dormant until the read lock has been acquired.

**Specified by:**

`lock` in interface `Lock`

## lockInterruptibly

```
public void lockInterruptibly()
                        throws InterruptedException
```

Acquires the read lock unless the current thread is interrupted.

Acquires the read lock if the write lock is not held by any thread and returns immediately.

If the write lock is held by any thread or the fairness policy prohibits acquisition of the read lock at this time, then the current thread becomes disabled for thread scheduling purposes and lies dormant until one of two things happens:

- The read lock is acquired by the current thread; or
- Some other thread interrupts the current thread.

If the current thread:

- has its interrupted status set on entry to this method; or
- is interrupted while acquiring the read lock,

then `InterruptedException` is thrown and the current thread's interrupted status is cleared.

In this implementation, as this method is an explicit interruption point, preference is given to responding to the interrupt over normal or reentrant acquisition of the lock.

**Specified by:**

`lockInterruptibly` in interface `Lock`

**Throws:**

`InterruptedException` - if the current thread is interrupted

### tryLock

```
public boolean tryLock()
```

Acquires the read lock only if the write lock is not held by another thread at the time of invocation.

Acquires the read lock if the write lock is not held by any thread and returns immediately with the value `true`. Even when this lock has been set to use a fair ordering policy, a call to `tryLock()` *will* immediately acquire the read lock if it is available, whether or not other threads are currently waiting for the read lock. This "barging" behavior can be useful in certain circumstances, even though it breaks fairness. If you want to honor the fairness setting for this lock, then use `tryLock(0, TimeUnit.SECONDS)` which is almost equivalent (it also detects interruption).

If the write lock is held by any thread then this method will return immediately with the value `false`.

**Specified by:**

`tryLock` in interface `Lock`

**Returns:**

`true` if the read lock was acquired

### tryLock

```
public boolean tryLock(long timeout,
                       TimeUnit unit)
                throws InterruptedException
```

Acquires the read lock if the write lock is not held by another thread within the given waiting time and the current thread has not been interrupted.

Acquires the read lock if the write lock is not held by another thread and returns immediately with the value `true`. If this lock has been set to use a fair ordering policy then an available lock *will not* be acquired if any other threads are waiting for the lock. This is in contrast to the `tryLock()` method. If you want a timed `tryLock` that does permit barging on a fair lock then combine the timed and un-timed forms together:

```
if (lock.tryLock() ||
    lock.tryLock(timeout, unit)) {
  ...
}
```

If the write lock is held by another thread then the current thread becomes disabled for thread scheduling purposes and lies dormant until one of three things happens:

- The read lock is acquired by the current thread; or
- Some other thread interrupts the current thread; or
- The specified waiting time elapses.

If the read lock is acquired then the value `true` is returned.

If the current thread:

- has its interrupted status set on entry to this method; or
- is interrupted while acquiring the read lock,

then `InterruptedException` is thrown and the current thread's interrupted status is cleared.

If the specified waiting time elapses then the value `false` is returned. If the time is less than or equal to zero, the method will not wait at all.

In this implementation, as this method is an explicit interruption point, preference is given to responding to the interrupt over normal or reentrant acquisition of the lock, and over reporting the elapse of the waiting time.

**Specified by:**

`tryLock` in interface `Lock`

**Parameters:**

`timeout` - the time to wait for the read lock

`unit` - the time unit of the timeout argument

**Returns:**

`true` if the read lock was acquired

**Throws:**

`InterruptedException` - if the current thread is interrupted

`NullPointerException` - if the time unit is null

## unlock

```
public void unlock()
```

Attempts to release this lock.

If the number of readers is now zero then the lock is made available for write lock attempts. If the current thread does not hold this lock then `IllegalMonitorStateException` is thrown.

**Specified by:**

`unlock` in interface `Lock`

**Throws:**

`IllegalMonitorStateException` - if the current thread does not hold this lock

## newCondition

```
public Condition newCondition()
```

Throws `UnsupportedOperationException` because `ReadLocks` do not support conditions.

**Specified by:**

`newCondition` in interface `Lock`

**Returns:**

A new `Condition` instance for this `Lock` instance

**Throws:**

`UnsupportedOperationException` - always

## toString

```
public String toString()
```

Returns a string identifying this lock, as well as its lock state. The state, in brackets, includes the String `"Read locks ="` followed by the number of held read locks.

**Overrides:**

`toString` in class `Object`

**Returns:**

a string identifying this lock, as well as its lock state

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.